Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Krishnamurthy A K **1BM24EC087**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

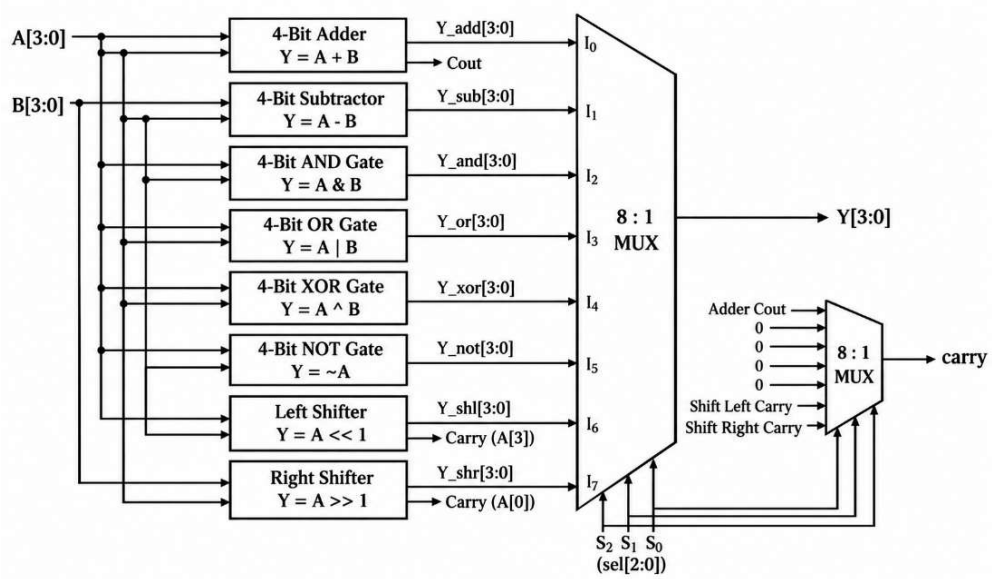
Table of Contents

Sl no.	Question	Pg no.
1	Arithmetic Logic Unit	1
2	4-Bit Comparator	4
3	4-Bit Parity Generator and Checker	6
4	4-Bit Barrel Shifter	8
5	Timer Module	10
6	Ring Counter	13
7	Johnson Counter	15
8a	Synchronous Counter	17
8b	Asynchronous Counter	19
9	4-Bit Shift Register	21
10	Divide-by-N Counter	23
11a	Mealy FSM — Sequence Detector for 101	25
11b	Moore FSM — Sequence Detector for 101	28
12	Random Sequence Generator Using LFSR	31

Question 1:

Arithmetic Logic Unit

Logic Circuit:



Design Code:

```
module alu (
    input [3:0] A, B,
    input [2:0] sel,
    output reg [3:0] Y,
    output reg carry
);

always @(*) begin
    Y = 0;
    carry = 0;

    case (sel)
        3'b000: {carry, Y} = {1'b0, A} + {1'b0, B};
```

```
        3'b001: Y = A - B;
        3'b010: Y = A & B;
        3'b011: Y = A | B;
        3'b100: Y = A ^ B;
        3'b101: Y = ~A;
        3'b110: begin
            Y = A << 1;
            carry = A[3];
        end
        3'b111: begin
            Y = A >> 1;
            carry = A[0];
        end
    endcase
end

endmodule
```

Testbench Code:

```
module tb_alu;

reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] Y;
wire carry;

alu uut(A, B, sel, Y, carry);

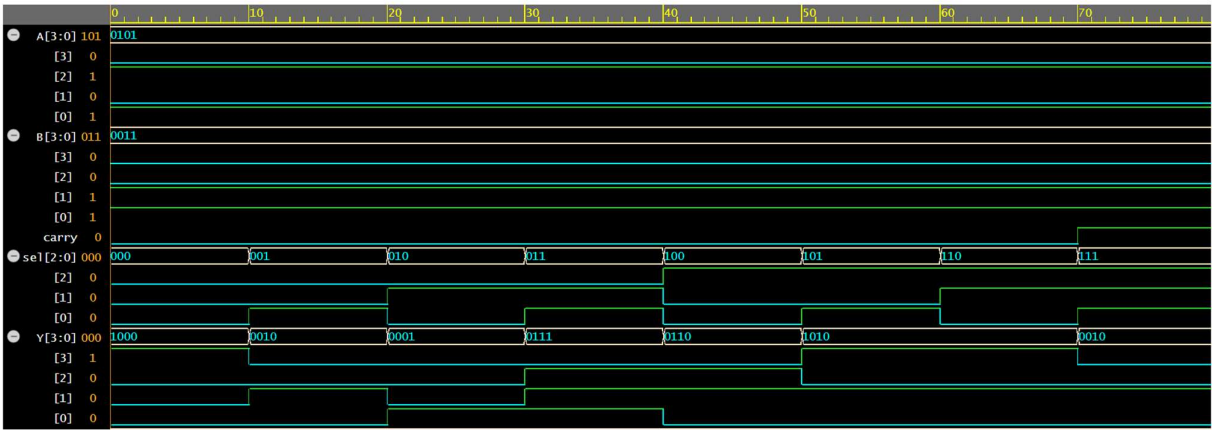
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_alu);
```

```
    A=5; B=3; sel=0; #10;
    sel=1; #10;
    sel=2; #10;
    sel=3; #10;
    sel=4; #10;
    sel=5; #10;
    sel=6; #10;
    sel=7; #10;

    $finish;
end

endmodule
```

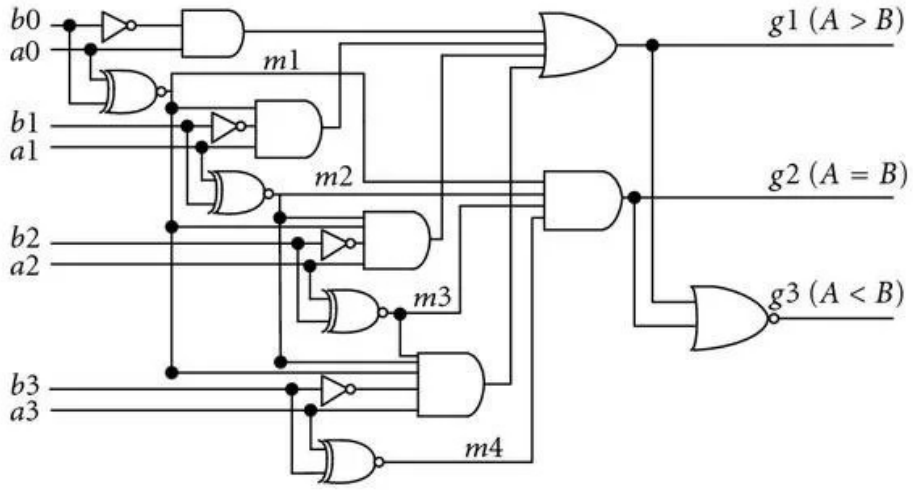
Output Waveform:



Question 2:

4-Bit Comparator

Logic Circuit:



Design Code:

```
module comparator (
   input [3:0] A, B,
   output A_gt_B,
   output A_eq_B,
   output A_lt_B
);

assign A_gt_B = (A > B);
assign A_eq_B = (A == B);
assign A_lt_B = (A < B);

endmodule
```

Testbench Code:

```
module tb_comparator;

reg [3:0] A, B;
wire A_gt_B, A_eq_B, A_lt_B;

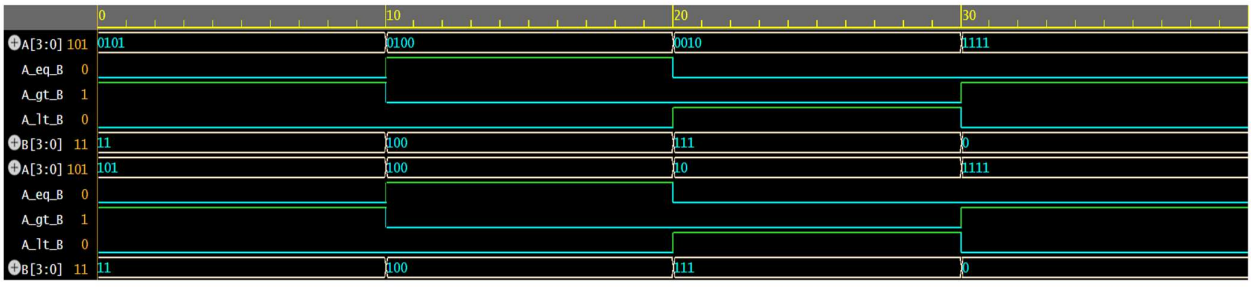
comparator uut(A, B, A_gt_B, A_eq_B, A_lt_B);

initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_comparator);

   A=5; B=3; #10;
   A=4; B=4; #10;
   A=2; B=7; #10;
   A=15; B=0; #10;

   $finish;
end
endmodule
```

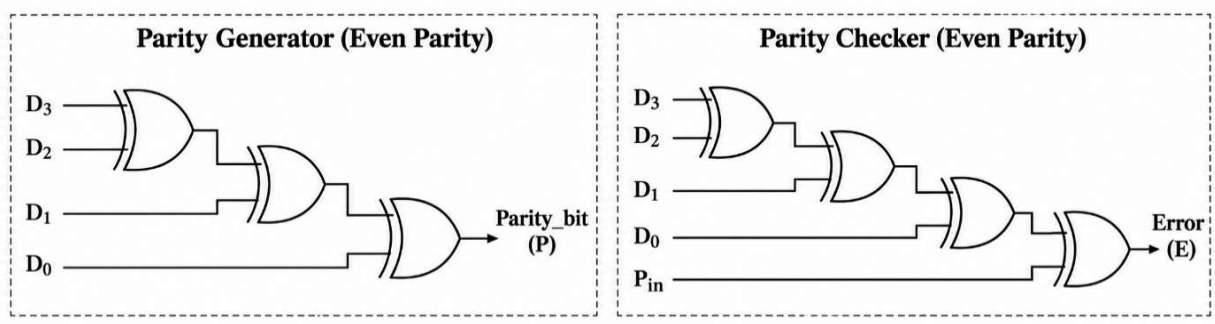
Output Waveform:



Question 3:

4-Bit Parity Generator and Checker

Logic Circuit:



Design Code:

```
module parity_generator_checker (
    input [3:0] data,
    input parity_in,
    output parity_out,
    output error
);

assign parity_out = ^data;
assign error = ^{data, parity_in};

endmodule
```

Testbench Code:

```
module tb_parity;

reg [3:0] data;
reg parity_in;
wire parity_out, error;

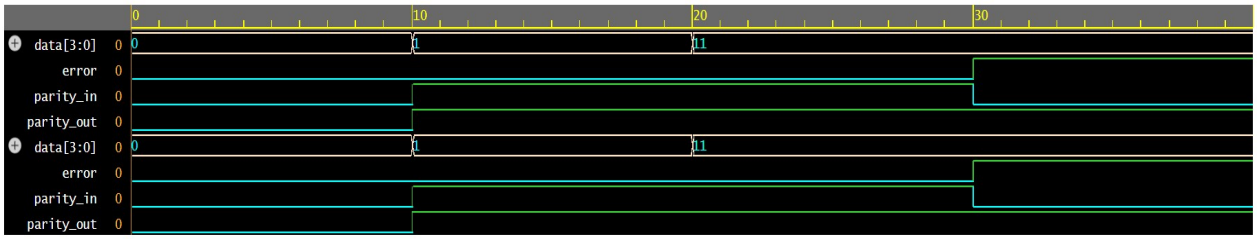
parity_generator_checker uut(data, parity_in, parity_out, error);

initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_parity);

   data=4'b0000; parity_in=0; #10;
   data=4'b0001; parity_in=1; #10;
   data=4'b1011; parity_in=1; #10;
   data=4'b1011; parity_in=0; #10;

   $finish;
end
endmodule
```

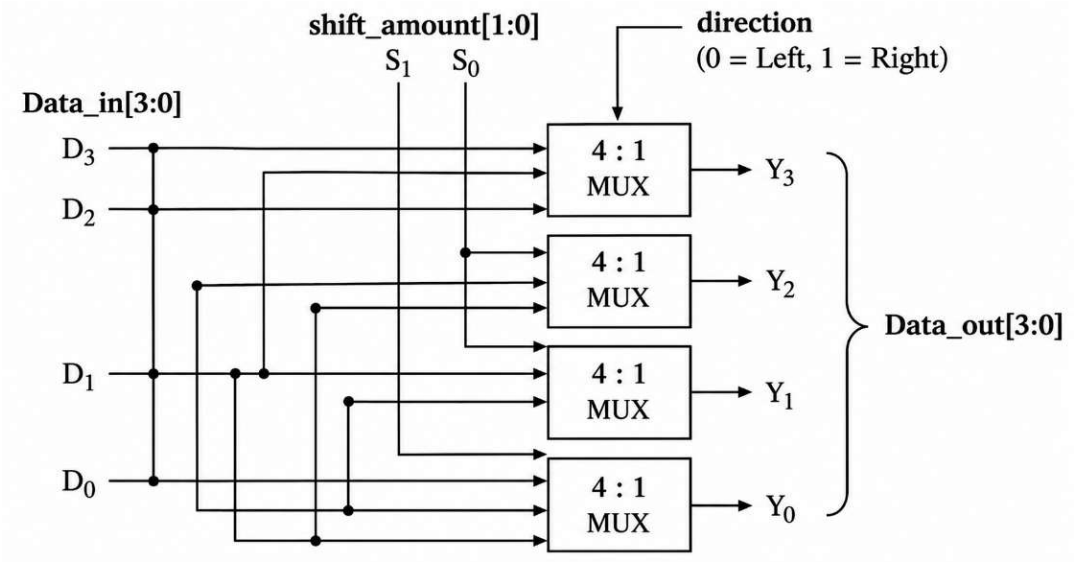
Output Waveform:



Question 4:

4-Bit Barrel Shifter

Logic Circuit:



Design Code:

```
module barrel_shifter (
    input [3:0] data_in,
    input [1:0] shift_amount,
    input direction,
    output reg [3:0] data_out
);

always @(*) begin
    if (direction == 1'b0)
        data_out = data_in << shift_amount;
    else
        data_out = data_in >> shift_amount;
end
endmodule
```

Testbench Code:

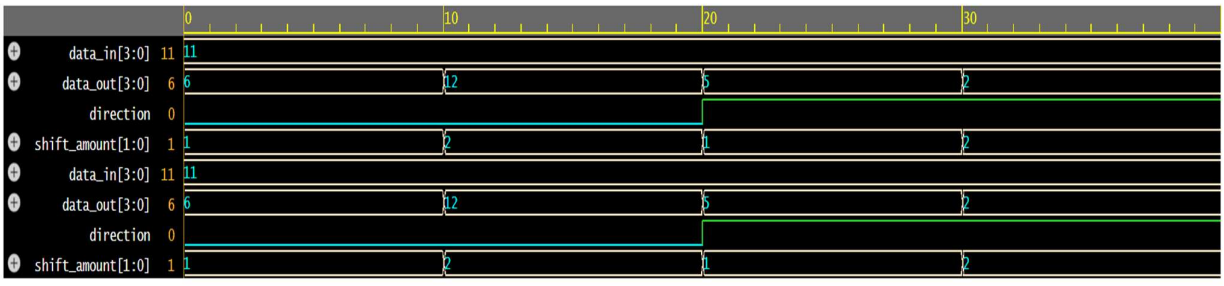
```
module tb_barrel_shifter;

reg [3:0] data_in;
reg [1:0] shift_amount;
reg direction;
wire [3:0] data_out;
barrel_shifter uut(data_in, shift_amount, direction, data_out);
initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_barrel_shifter);

   data_in=4'b1011; shift_amount=1; direction=0; #10;
   data_in=4'b1011; shift_amount=2; direction=0; #10;
   data_in=4'b1011; shift_amount=1; direction=1; #10;
   data_in=4'b1011; shift_amount=2; direction=1; #10;

   $finish;
end
endmodule
```

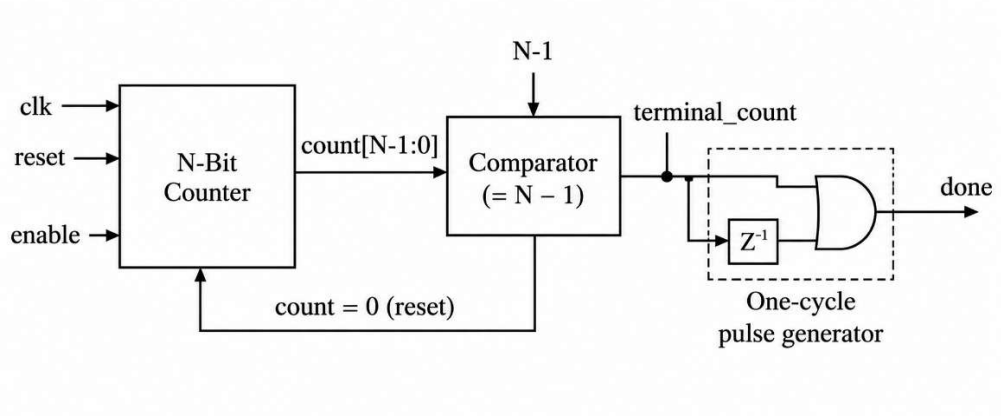
Output Waveform:



Question 5:

Timer Module

Logic Circuit:



Design Code:

```
module timer #(
   parameter N = 5
)(
   input clk,
   input reset,
   input enable,
   output reg done
);

reg [31:0] count;

always @(posedge clk or posedge reset) begin
   if (reset) begin
      count <= 0;
      done <= 0;
   end

 else if (enable) begin
```

```
        if (count == N-1) begin
            count <= 0;
            done <= 1;
        end
        else begin
            count <= count + 1;
            done <= 0;
        end
    end
    else begin
        done <= 0;
    end
end
endmodule
```

Testbench Code:

```
module tb_timer;

reg clk, reset, enable;
wire done;

timer #(.N(5)) uut(clk, reset, enable, done);

always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_timer);

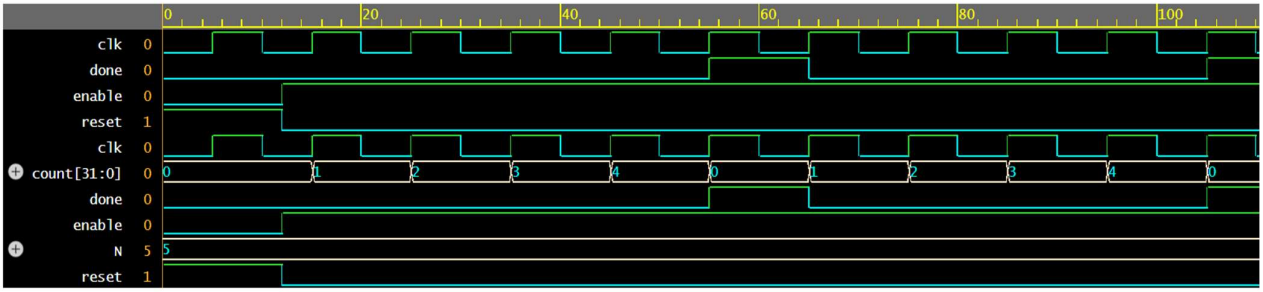
    clk=0;
    reset=1;
```

```
    enable=0;

    #12 reset=0;
    enable=1;

    #100 enable=0;
    #20 $finish;
end
endmodule
```

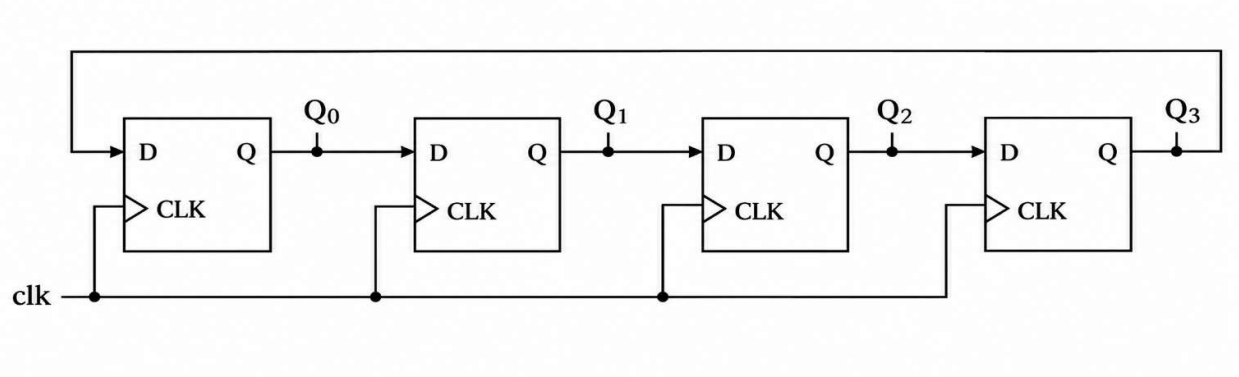
Output Waveform:



Question 6:

Ring Counter

Logic Circuit:



Design Code:

```
module ring_counter (
   input clk,
   input reset,
   output reg [3:0] q
);

always @(posedge clk or posedge reset) begin
   if (reset)
      q <= 4'b0001;
   else
      q <= {q[2:0], q[3]};
end

endmodule
```

Testbench Code:

```
module tb_ring_counter;

reg clk, reset;
wire [3:0] q;

ring_counter uut(clk, reset, q);

always #5 clk = ~clk;

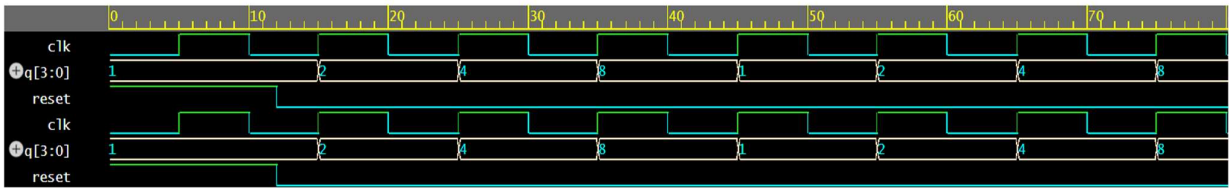
initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_ring_counter);

   clk=0;
   reset=1;

   #12 reset=0;
   #80 $finish;
end


endmodule
```

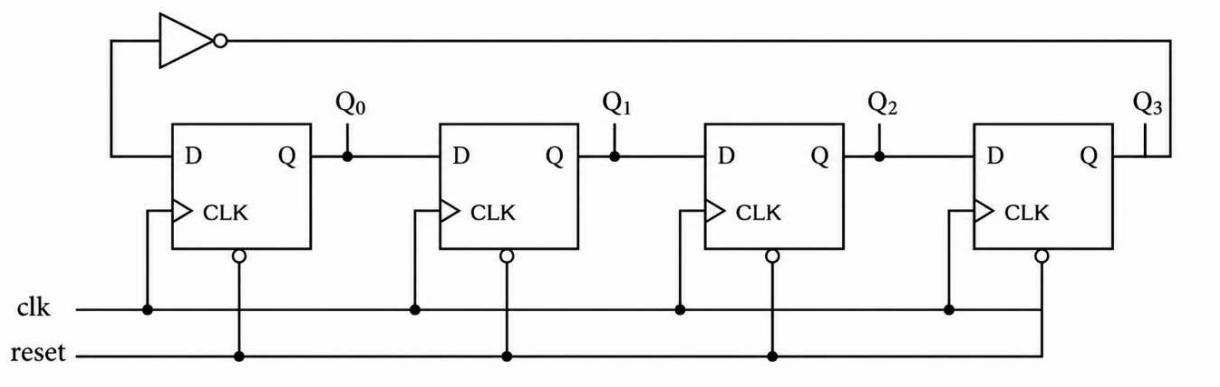
Output Waveform:



Question 7:

Johnson Counter

Logic Circuit:



Design Code:

```
module johnson_counter (
    input clk,
    input reset,
    output reg [3:0] q
);

always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 4'b0000;
    else
        q <= {q[2:0], ~q[3]};
end

endmodule
```

Testbench Code:

```
module tb_johnson_counter;

reg clk, reset;
wire [3:0] q;

johnson_counter uut(clk, reset, q);

always #5 clk = ~clk;

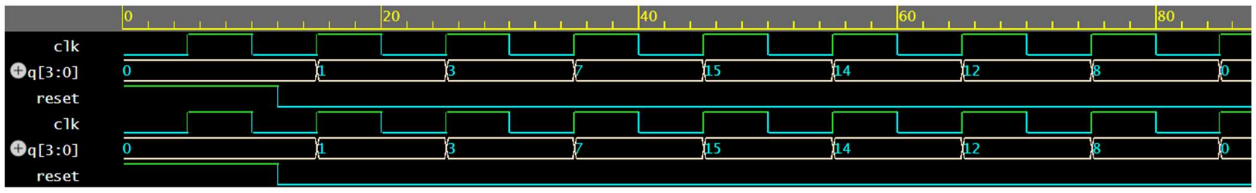
initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_johnson_counter);

   clk=0;
   reset=1;

   #12 reset=0;
   #100 $finish;
end

endmodule
```

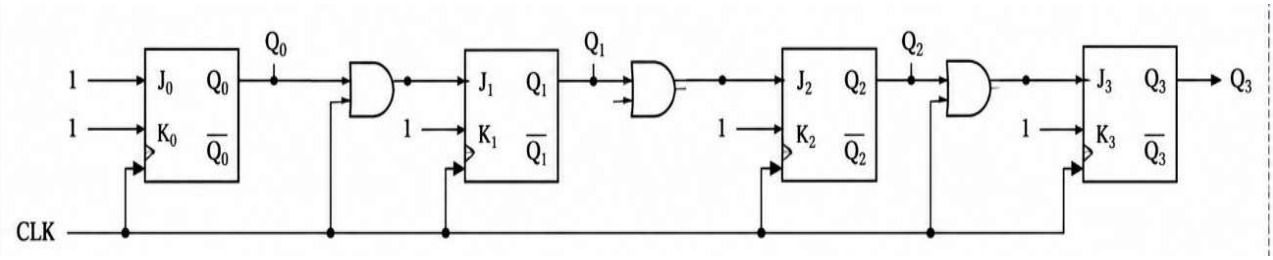
Output Waveform:



Question 8a:

Synchronous Counter

Logic Circuit:



Design Code:

```
module synchronous_counter (
   input clk,
   input reset,
   output reg [3:0] q
);

always @(posedge clk or posedge reset) begin
   if (reset)
      q <= 4'b0000;
   else
      q <= q + 1'b1;
end

endmodule
```

Testbench Code:

```
module tb_synchronous_counter;

reg clk, reset;
wire [3:0] q;

synchronous_counter uut(clk, reset, q);

always #5 clk = ~clk;

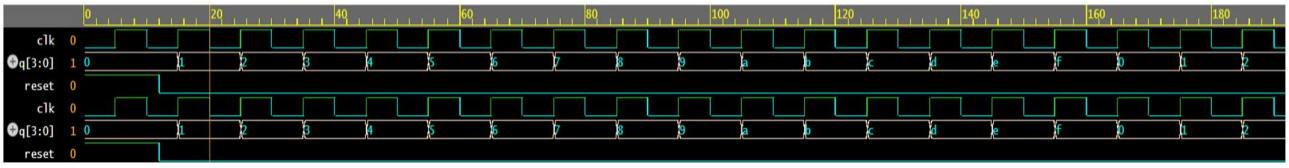
initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_synchronous_counter);

   clk=0;
   reset=1;

   #12 reset=0;
   #180 $finish;
end

endmodule
```

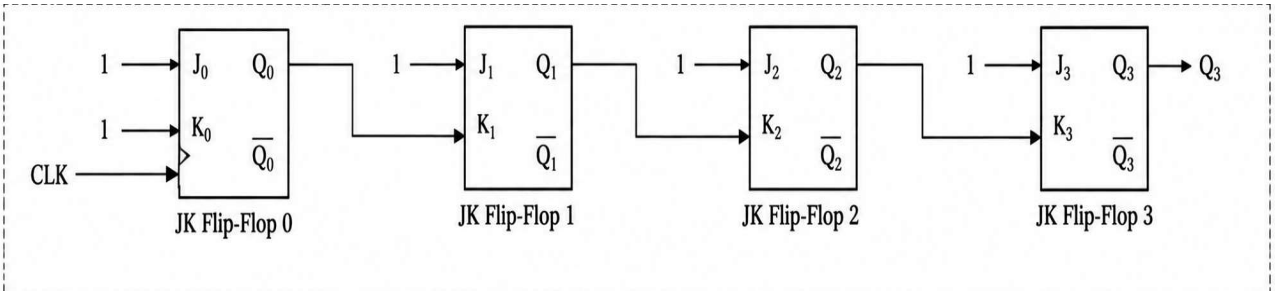
Output Waveform:



Question 8b:

Asynchronous Counter

Logic Circuit:



Design Code:

```
module asynchronous_counter (
   input clk,
   input reset,
   output reg [3:0] q
);

always @(posedge clk or posedge reset) begin
   if (reset)
      q[0] <= 0;
   else
      q[0] <= ~q[0];
end

always @(negedge q[0] or posedge reset) begin
   if (reset)
      q[1] <= 0;
   else
      q[1] <= ~q[1];
end

always @(negedge q[1] or posedge reset) begin
   if (reset)
      q[2] <= 0;
   else
      q[2] <= ~q[2];
end
```

```
always @(negedge q[2] or posedge reset) begin
   if (reset)
      q[3] <= 0;
   else
      q[3] <= ~q[3];
end

endmodule
```

Testbench Code:

```
module tb_asynchronous_counter;

reg clk, reset;
wire [3:0] q;

asynchronous_counter uut(clk, reset, q);

always #5 clk = ~clk;

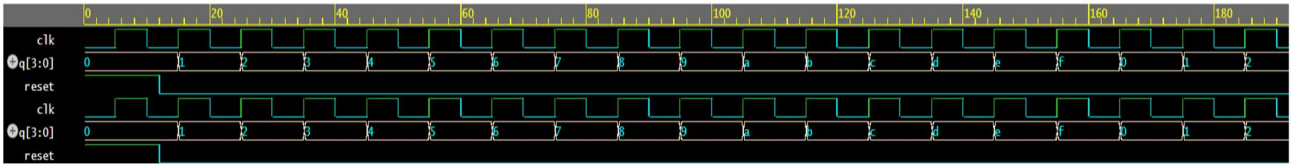
initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_asynchronous_counter);

   clk=0;
   reset=1;

   #12 reset=0;
   #180 $finish;
end

endmodule
```

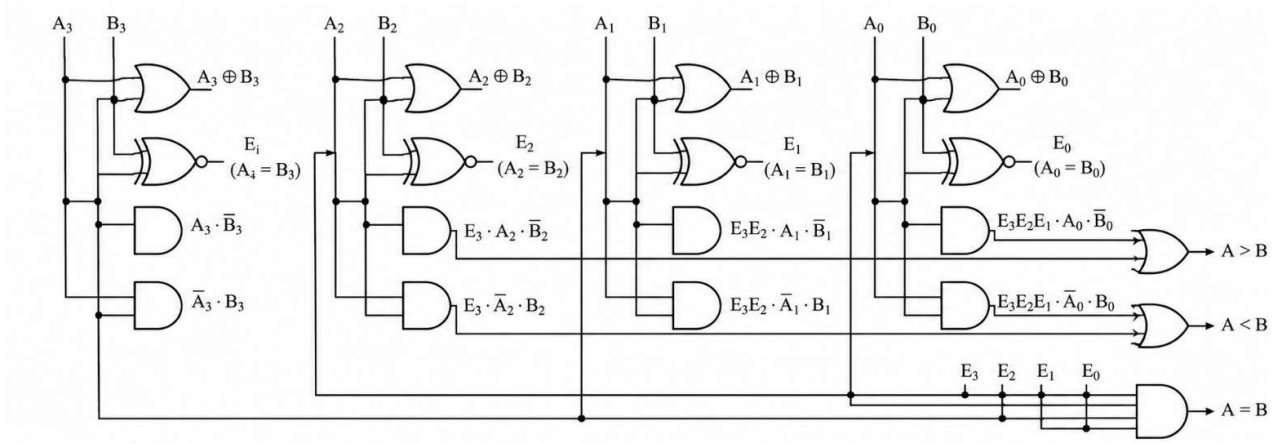
Output Waveform:



Question 9:

4-Bit Shift Register

Logic Circuit:



Design Code:

```
module shift_register (
   input clk,
   input reset,
   input serial_in,
   output serial_out,
   output reg [3:0] q
);

assign serial_out = q[3];

always @(posedge clk or posedge reset) begin
   if (reset)
      q <= 4'b0000;
   else
      q <= {q[2:0], serial_in};
end

endmodule
```

Testbench Code:

```
module tb_shift_register;

reg clk, reset, serial_in;
wire serial_out;
wire [3:0] q;
shift_register uut(clk, reset, serial_in, serial_out, q);

always #5 clk = ~clk;

initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_shift_register);

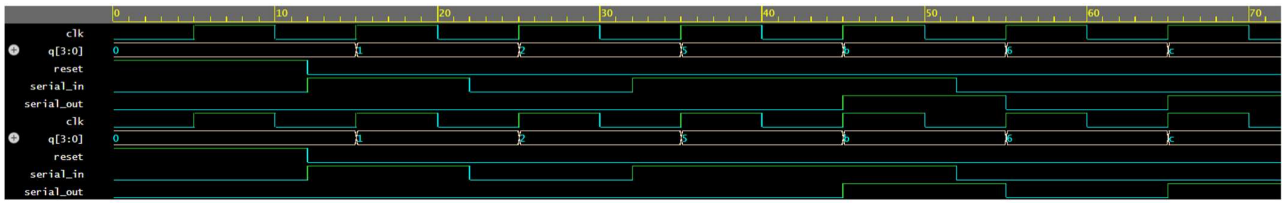
   clk=0;
   reset=1;
   serial_in=0;
   #12 reset=0;

   serial_in=1; #10;
   serial_in=0; #10;
   serial_in=1; #10;
   serial_in=1; #10;
   serial_in=0; #10;
   serial_in=0; #10;

   $finish;
end

endmodule
```

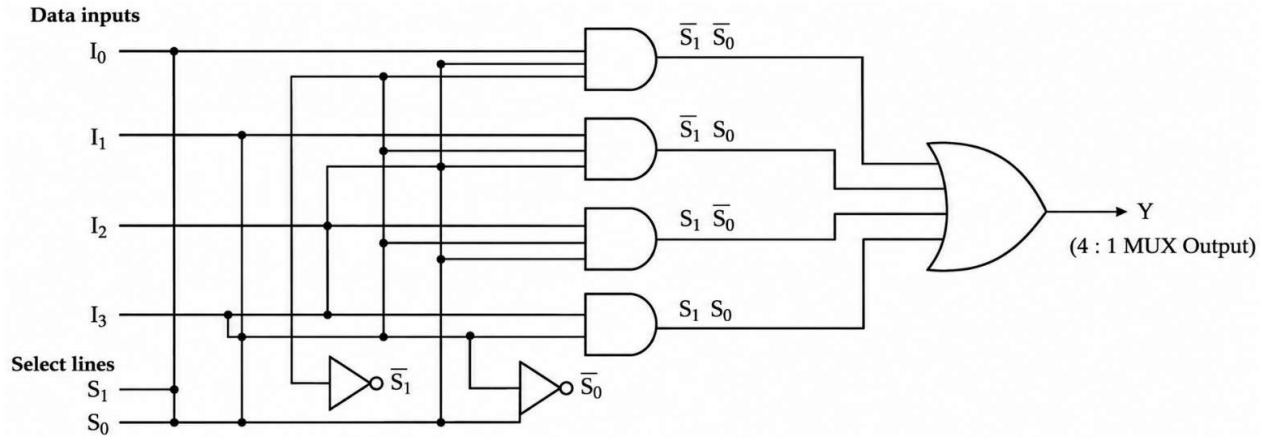
Output Waveform:



Question 10:

Divide-by-N Counter

Logic Circuit:



Design Code:

```
module divide_by_n #(
   parameter N = 5
)(
   input clk,
   input reset,
   output reg clk_out
);

localparam WIDTH = (N <= 1) ? 1 : $clog2(N);
reg [WIDTH-1:0] count;

always @(posedge clk or posedge reset) begin
   if (reset) begin
      count <= 0;
      clk_out <= 0;
   end
   else if (count == N-1) begin
      count <= 0;
      clk_out <= ~clk_out;
   end
   else begin
      count <= count + 1'b1;
   end
```

```
end

endmodule
```

Testbench Code:

```
module tb_divide_by_n;

reg clk, reset;
wire clk_out;

divide_by_n #(.N(5)) uut(clk, reset, clk_out);

always #5 clk = ~clk;

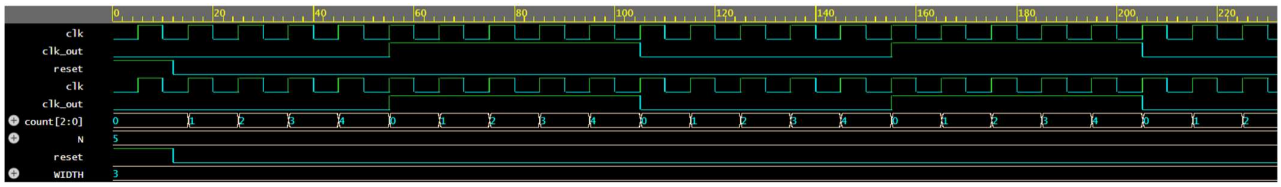
initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_divide_by_n);

   clk=0;
   reset=1;

   #12 reset=0;
   #220 $finish;
end

endmodule
```

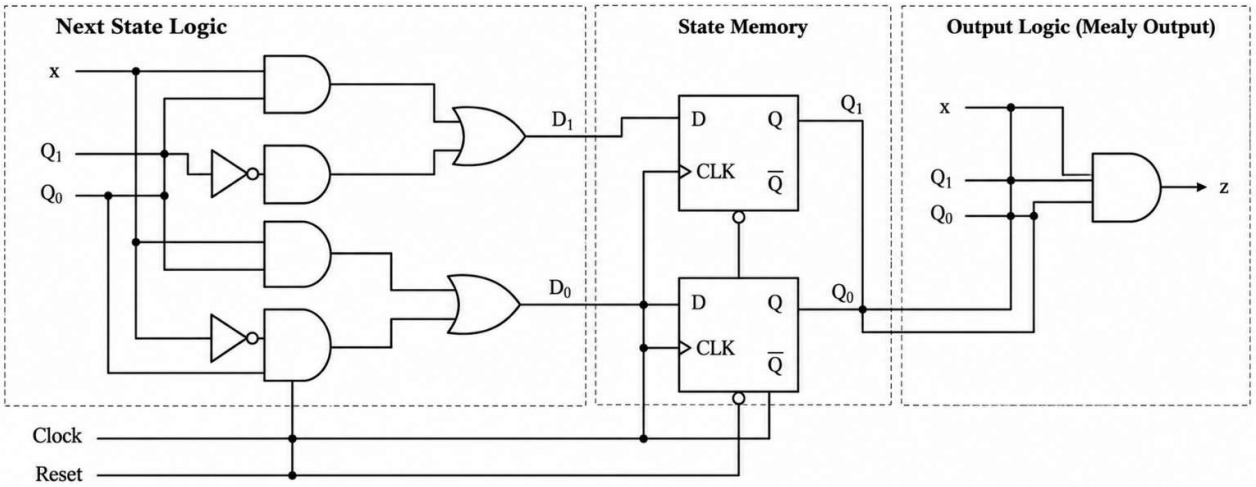
Output Waveform:



Question 11a:

Mealy FSM — Sequence Detector for 101

Logic Circuit:



Design Code:

```
module mealy_detector (
   input clk,
   input reset,
   input x,
   output reg z
);

reg [1:0] state, next_state;

localparam S0 = 2'b00,
           S1 = 2'b01,
           S2 = 2'b10;

always @(posedge clk or posedge reset) begin
   if (reset)
      state <= S0;
   else
      state <= next_state;
end

always @(*) begin
```

```
    next_state = state;
    z = 0;

    case (state)
        S0: begin
            if (x)
                next_state = S1;
            else
                next_state = S0;
        end

        S1: begin
            if (x)
                next_state = S1;
            else
                next_state = S2;
        end

        S2: begin
            if (x) begin
                next_state = S1;
                z = 1;
            end
            else
                next_state = S0;
        end

        default: next_state = S0;
    endcase
end

endmodule
```

Testbench Code:

```
module tb_mealy_detector;

reg clk, reset, x;
wire z;

mealy_detector uut(clk, reset, x, z);

always #5 clk = ~clk;
```

```
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_mealy_detector);

    clk=0;
    reset=1;
    x=0;

    #12 reset=0;

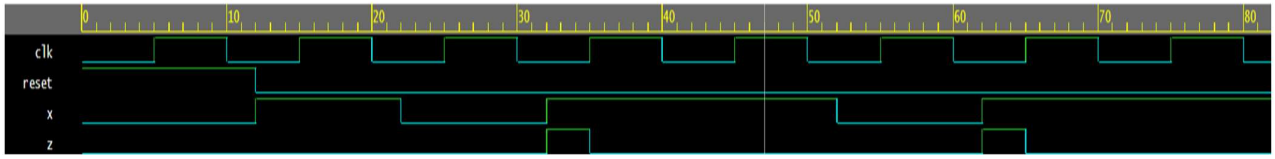
    x=1; #10;
    x=0; #10;
    x=1; #10;

    x=1; #10;
    x=0; #10;
    x=1; #10;

    #10 $finish;
end

endmodule
```

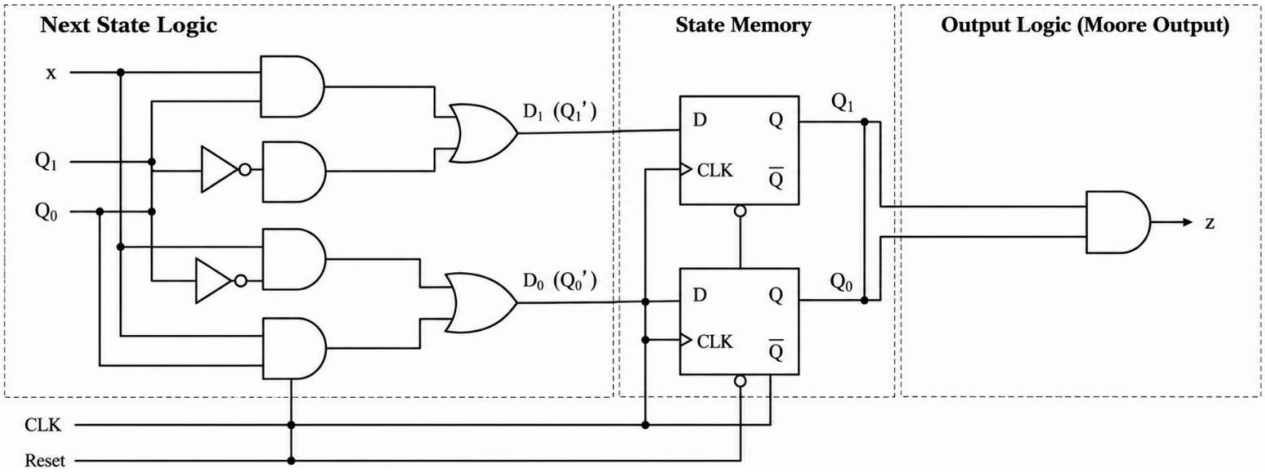
Output Waveform:



Question 11b:

Moore FSM — Sequence Detector for 101

Logic Circuit:



Design Code:

```
module moore_detector (
   input clk,
   input reset,
   input x,
   output reg z
);

reg [1:0] state, next_state;

localparam S0 = 2'b00,
           S1 = 2'b01,
           S2 = 2'b10,
           S3 = 2'b11;

always @(posedge clk or posedge reset) begin
   if (reset)
      state <= S0;
   else
      state <= next_state;
end
```

```
always @(*) begin
   next_state = S0;

   case (state)
      S0: next_state = x ? S1 : S0;
      S1: next_state = x ? S1 : S2;
      S2: next_state = x ? S3 : S0;
      S3: next_state = x ? S1 : S2;
      default: next_state = S0;
   endcase
end

always @(*) begin
   z = (state == S3);
end

endmodule
```

Testbench Code:

```
module tb_moore_detector;

reg clk, reset, x;
wire z;

moore_detector uut(clk, reset, x, z);

always #5 clk = ~clk;

initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_moore_detector);

   clk=0;
   reset=1;
   x=0;

   #12 reset=0;

   x=1; #10;
   x=0; #10;
```

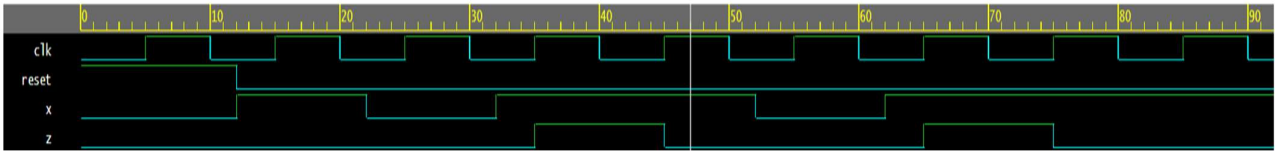
```
    x=1; #10;

    x=1; #10;
    x=0; #10;
    x=1; #10;

    #20 $finish;
end

endmodule
```

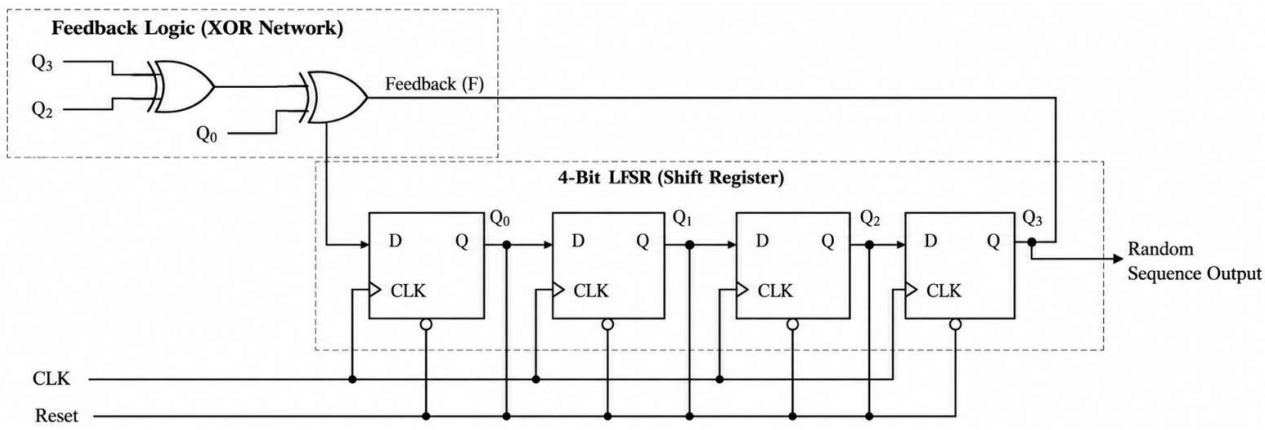
Output Waveform:



Question 12:

Random Sequence Generator Using LFSR

Logic Circuit:



Design Code:

```
module lfsr (
   input clk,
   input reset,
   output reg [3:0] q
);

wire feedback;
assign feedback = q[3] ^ q[2];

always @(posedge clk or posedge reset) begin
   if (reset)
      q <= 4'b0001;
   else
      q <= {q[2:0], feedback};
end

endmodule
```

Testbench Code:

```
module tb_lfsr;

reg clk, reset;
wire [3:0] q;

lfsr uut(clk, reset, q);

always #5 clk = ~clk;

initial begin
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_lfsr);

   clk=0;
   reset=1;

   #12 reset=0;
   #180 $finish;
end

endmodule
```

Output Waveform:

